



Rangamati Science and Technology University

Department of Computer Science and Engineering

Course Code: CSE 3104

Course Title: Operating System Lab

ShellGuard: Syscall Sandbox with seccomp-BPF

A Project Report for CSE 3104: Operating System Lab

Submitted by:

Mohammad Tawheedul Islam (ID: 2201011025)
MD. Abdur Rahman Chowdhury (ID: 2201011033)
Jobayer Hossain Khandaker (ID: 2201011036)

Under the guidance of

Md Mynoddin
Assistant Professor, Dept. of CSE, RMSTU

Session: 2022–2023
Submission Date: 15 June, 2026

Abstract

This project presents **ShellGuard**, a command-line syscall sandboxing tool built on Linux seccomp-BPF and the libseccomp library. ShellGuard reads a human-readable policy file specifying which system calls (syscalls) a target program is permitted to invoke, constructs a default-deny seccomp-BPF filter from this policy, and applies it to a forked child process before executing the target program. If the target program attempts a denied syscall, the Linux kernel immediately terminates it with **SIGSYS** (signal 31, “Bad system call”), preventing the harmful operation from completing. ShellGuard was validated against a benign shell script (which ran successfully), a script attempting network access via **curl** (which was blocked), and a set of C test programs exercising file I/O and process creation under both permissive and strict policies. All five test cases produced the expected outcomes, confirming correct enforcement of allow and deny rules. The tool also logs every run’s result to a persistent file, providing an audit trail of sandboxed executions. The complete source code and test artifacts are available at <https://github.com/tawheedul-islam/shellguard>.

Keywords: operating systems, system calls, sandboxing, seccomp-BPF, privilege restriction, Linux security, process isolation

Contents

1	Introduction	3
1.1	Motivation and Background	3
1.2	Problem Statement	3
1.3	Contributions	3
1.4	Report Organization	4
2	Related Work	4
3	Methodology	5
3.1	System Architecture	5
3.2	Algorithms and Data Structures	7
3.3	Implementation Details	8
3.4	Datasets / Traces / Workloads	8
3.5	Evaluation Metrics	9
4	Implementation and Experimental Setup	9
4.1	Hardware and Software Environment	9
4.2	Baseline for Comparison	10
4.3	Experimental Procedure	10
5	Results and Discussion	11
5.1	Quantitative Results	11
5.2	Analysis and Interpretation	11
5.2.1	Benign Script Execution	11
5.2.2	Malicious Script Execution (Network Access)	12
5.2.3	File I/O Test	13
5.2.4	Process Creation Test (fork/clone)	13
5.3	Limitations and Threats to Validity	14
6	Conclusion and Future Work	15
A	Code Repository and Artifacts	16
B	Additional Data or Raw Logs	16

1 Introduction

1.1 Motivation and Background

Modern operating systems run a wide variety of programs with varying levels of trust. A single vulnerable or malicious program can compromise an entire system if it is allowed unrestricted access to kernel resources through system calls (syscalls). Syscalls are the only interface through which user-space programs request services from the kernel, such as file access, network communication, process creation, and memory management. Restricting which syscalls a program is permitted to invoke is therefore an effective way to limit potential damage, even if the program contains bugs or malicious code.

seccomp-BPF (secure computing mode with Berkeley Packet Filter) is a Linux kernel feature that allows a process to install a custom filter defining exactly which syscalls are permitted. This mechanism is widely used in real-world software, including web browsers (Chrome, Firefox), container runtimes (Docker, runc), and sandboxing frameworks. Despite its widespread use, seccomp-BPF is often seen as complex and low-level. This project demonstrates that a functional syscall sandbox can be built with a relatively small amount of C code using the libseccomp library.

1.2 Problem Statement

This project aims to design and implement a simple command-line sandboxing tool, **Shell-Guard**, that enforces a configurable syscall policy on an arbitrary target program. The tool must:

- Read a human-readable policy file specifying allowed and denied syscalls.
- Apply a seccomp-BPF filter to a forked child process before executing the target program.
- Report whether the program exited normally or was killed due to a policy violation (SIGSYS).
- Log all run results to a persistent file for auditing.

1.3 Contributions

The main contributions of this project are:

- A working CLI sandboxing tool (**shellguard**) that enforces configurable syscall policies using seccomp-BPF and libseccomp.
- A simple, human-readable policy file format (**ALLOW/ DENY** rules) that can be written and modified without any special tooling.
- Two policy files: a permissive policy for running bash scripts safely, and a strict policy additionally denying process creation.

- A test corpus of shell scripts and C programs demonstrating both correct enforcement of allowed operations and correct blocking of denied syscalls.
- An iterative methodology for building a working seccomp policy using **strace** to identify required syscalls.

1.4 Report Organization

The remainder of this report is organized as follows. Section 2 reviews related work on syscall sandboxing and existing tools. Section 3 describes the design and methodology of ShellGuard. Section 4 covers the implementation details and experimental setup. Section 5 presents and discusses the test results. Section 6 concludes the report and outlines future work.

2 Related Work

Syscall sandboxing and privilege restriction have been active areas of research and development in operating systems security. This section reviews the most relevant existing approaches and tools.

Standard OS Security Mechanisms

Traditional Unix systems rely on user/group permissions and the `setuid/setgid` mechanism to limit program privileges. While effective for file-level access control, these mechanisms do not restrict which kernel interfaces (syscalls) a program may invoke. A program running as a non-root user can still call `socket()`, `ptrace()`, or `fork()` freely, which leaves significant attack surface exposed.

seccomp and seccomp-BPF

The **seccomp** mechanism was introduced in Linux 2.6.12 (2005) as a strict mode that limited a process to only four syscalls. The more flexible **seccomp-BPF** extension, introduced in Linux 3.5 (2012), allows processes to install custom BPF filter programs that inspect syscall numbers and arguments. This is the mechanism underlying ShellGuard. Writing raw BPF bytecode by hand is error-prone; the **libseccomp** library [1] provides a higher-level C API that compiles human-readable rule specifications into BPF programs automatically.

Existing Sandboxing Tools

Several production-grade tools use seccomp-BPF internally:

- **Docker/runc**: Container runtimes apply a default seccomp profile that blocks approximately 44 syscalls considered dangerous for containerized workloads [2]. Profiles are specified as JSON files.

- **Firejail**: A Linux sandboxing tool that combines seccomp-BPF with Linux namespaces and capabilities to sandbox desktop applications.
- **Chromium/Firefox**: Both browsers apply seccomp-BPF filters to renderer processes to limit the syscall surface exposed to potentially malicious web content.
- **Minijail**: A Google tool (used in Chrome OS and Android) for sandboxing processes using seccomp, namespaces, and cgroups.

eBPF-Based Observability

A related technology, **eBPF** (extended BPF), allows user-space programs to attach programs to kernel tracepoints, including syscall entry/exit events, for observability and enforcement purposes. Tools such as **Falco** and **Tetragon** use eBPF to detect anomalous syscall behavior at runtime. Unlike seccomp-BPF (which is applied per-process and enforces static policies), eBPF-based tools can make dynamic, context-aware decisions and monitor entire system activity. Project 16 in this course (eBPF Explorer) explores this approach.

What ShellGuard Adds

Existing tools such as Docker and Firejail are powerful but complex, and their seccomp profiles are maintained by security experts. No simple, beginner-accessible CLI tool exists that allows a user to write a plain-text **ALLOW/DENY** policy file and immediately sandbox an arbitrary program with it. ShellGuard fills this gap: it is a minimal, self-contained implementation that demonstrates the core seccomp-BPF workflow (policy parsing, filter construction, fork/exec sandboxing, and result logging) in approximately 150 lines of C code, making it suitable as an educational tool for understanding OS-level privilege restriction.

3 Methodology

3.1 System Architecture

ShellGuard follows a four-stage pipeline architecture. Figure 1 illustrates the overall flow.

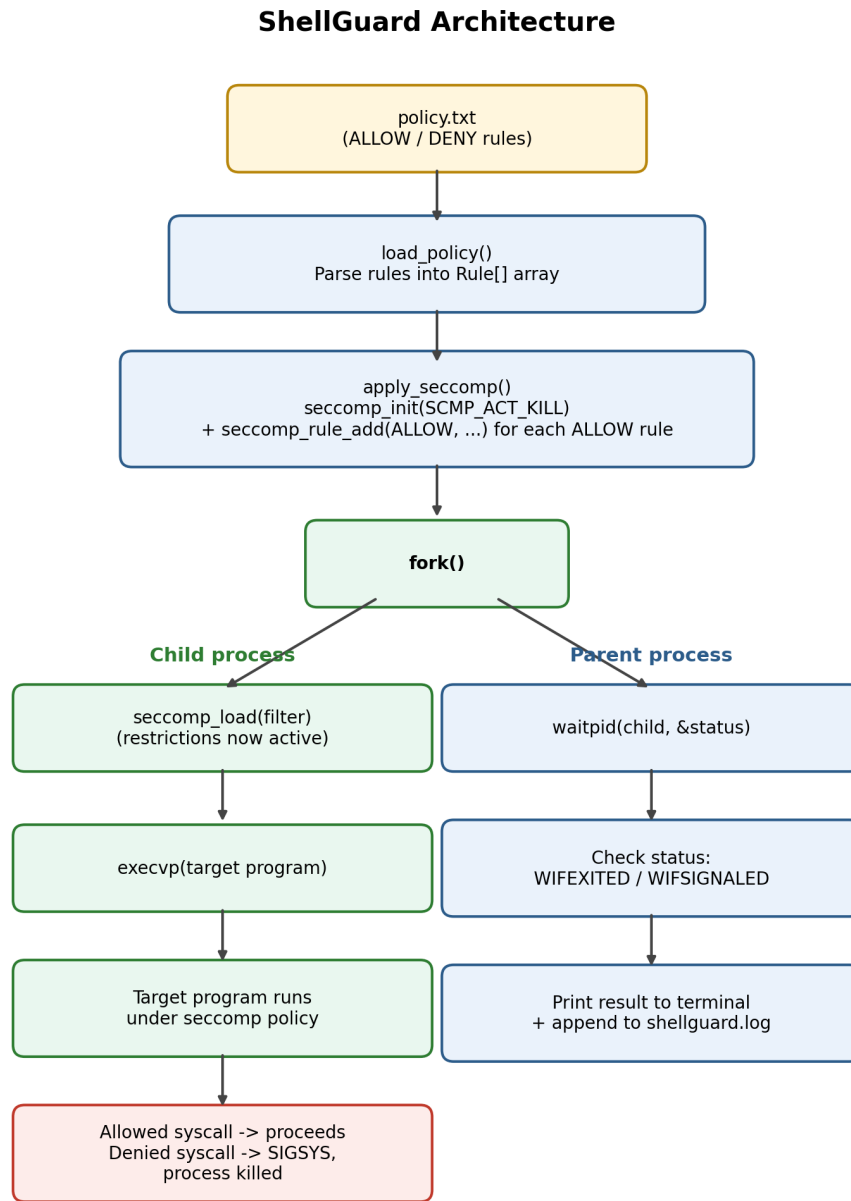


Figure 1: ShellGuard architecture: the policy file is parsed into rules, compiled into a seccomp-BPF filter, and applied to the child process before it executes the target program.

The four stages are:

1. **Policy Loading:** ShellGuard reads a plain-text policy file containing ALLOW/DENY rules, one per line. Comments (lines starting with #) and blank lines are ignored. Rules are stored in memory as an array of (syscall name, allow/deny) pairs.

2. **Filter Construction:** A seccomp filter context is initialized with a default action of `SCMP_ACT_KILL` (deny everything by default). For each `ALLOW` rule, the syscall name is resolved to its numeric ID using `seccomp_syscall_resolve_name()`, and an `SCMP_ACT_ALLOW` rule is added. `DENY` rules require no explicit action since the default already blocks them.
3. **Sandboxed Execution:** ShellGuard calls `fork()`. The child process loads the seccomp filter (`seccomp_load()`) and immediately calls `execvp()` to replace itself with the target program. Because seccomp filters persist across `execve()`, the target program runs under the full restrictions of the policy.
4. **Result Reporting:** The parent calls `waitpid()` and inspects the exit status. If the child exited normally, the exit code is reported. If it was killed by `SIGSYS` (signal 31), this is reported as a seccomp policy violation. All results are also appended to `shellguard.log`.

3.2 Algorithms and Data Structures

The core data structure is a simple array of `Rule` structs:

```
typedef struct {
    char syscall_name[64];
    int allow; // 1 = ALLOW, 0 = DENY
} Rule;

Rule rules[MAX_RULES]; // array of up to 128 rules
```

Policy loading follows a straightforward line-by-line parsing algorithm:

Input: policy filename, max_rules

Output: array of `Rule` structs, count of rules

open file;

while *lines remain and count < max_rules* **do**

 read line;

if *line is comment or blank* **then**

 skip;

end

 parse ACTION and SYSCALL_NAME from line;

 store in rules[count], set allow flag;

 increment count;

end

close file;

return count;

Algorithm 1: load_policy()

Filter construction follows the default-deny principle:


```

Input: rules array, count
ctx ← seccomp_init(SCMP_ACT_KILL);
for each rule in rules do
    if rule.allow == 1 then
        num ← seccomp_syscall_resolve_name(rule.syscall_name);
        if num ≠ error then
            seccomp_rule_add(ctx, SCMP_ACT_ALLOW, num);
        end
    end
end
seccomp_load(ctx);
seccomp_release(ctx);

```

Algorithm 2: `apply_seccomp()`

3.3 Implementation Details

- **Programming language:** C (C99), chosen for direct access to Linux system call interfaces and compatibility with libseccomp.
- **Key library:** libseccomp (version available in Ubuntu 24.04 via `apt install libseccomp-dev`), which provides a high-level API for constructing seccomp-BPF filters without writing raw BPF bytecode.
- **Build:** Single-file compilation with no build system required:

```
gcc shellguard.c -o shellguard -lseccomp
```
- **No modifications** to the Linux kernel or any existing codebase were required; ShellGuard operates entirely in user space.
- **Source code** is available at <https://github.com/tawheedul-islam/shellguard>.

3.4 Datasets / Traces / Workloads

ShellGuard does not require external datasets or traces. The test workloads were created specifically for this project:

- **benign.sh** – a bash script that prints two lines of text using `echo`. Exercises only basic I/O and process startup syscalls.
- **malicious.sh** – a bash script that attempts to contact an external server using `curl`. Exercises network syscalls (`socket`, `connect`) that are denied by `policy.txt`.
- **fileio_test.c** – a C program that opens a file for writing, writes a line, and closes it. Exercises `openat`, `write`, `close`.
- **fork_test.c** – a C program that calls `fork()` to create a child process. Exercises the `clone` syscall (the Linux implementation of `fork()`).

Two policy files were used:

- **policy.txt** – allows all syscalls required by bash and standard C programs (≈ 58 syscalls), while denying `socket`, `connect`, and `ptrace`.
- **policy_strict.txt** – same as above but additionally denies `clone` and `fork`, blocking process creation entirely.

3.5 Evaluation Metrics

Since ShellGuard is a security enforcement tool rather than a performance optimization, the primary evaluation metric is **correctness**:

- **Allow correctness:** Does a program that uses only permitted syscalls run to completion with exit code 0?
- **Deny correctness:** Does a program that attempts a denied syscall get terminated with SIGSYS (signal 31)?
- **Log accuracy:** Is every run’s outcome correctly recorded in `shellguard.log`?

Secondary verification was performed using `strace` to confirm that SIGSYS was triggered by the specific denied syscall (e.g., `connect` for network access, `clone` for process creation).

4 Implementation and Experimental Setup

4.1 Hardware and Software Environment

All development and testing was performed on a single machine with the following configuration:

Table 1: Hardware and software environment

Component	Details
Host OS	Windows 11 Pro
Virtualization	VMware Workstation
Guest OS	Ubuntu 24.04 LTS (64-bit)
Kernel	Linux (x86_64)
Compiler	gcc (Ubuntu, default version)
Library	libseccomp-dev (via apt)
Tools	strace, git, nano
Version Control	Git + GitHub

ShellGuard requires no special hardware. It runs on any standard x86_64 Linux system with kernel version 3.5 or later (seccomp-BPF support) and libseccomp installed.

4.2 Baseline for Comparison

ShellGuard is a sandboxing enforcement tool, not a performance optimization, so there is no numerical baseline to compare against. Instead, the baseline is defined as **unsandboxed execution**: running the same target programs directly (without ShellGuard) to confirm that they work correctly in an unrestricted environment.

- **benign.sh** run directly: prints text, exits 0.
- **malicious.sh** run directly: `curl` successfully contacts `example.com` and exits 0 (network access is unrestricted without ShellGuard).
- **fileio_test** run directly: writes file, exits 0.
- **fork_test** run directly: forks successfully, both processes print and exit 0.

This baseline confirms that all test programs function correctly without sandboxing, and that any failures observed under ShellGuard are caused by the seccomp policy, not by bugs in the test programs themselves.

4.3 Experimental Procedure

The experimental procedure for each test case was as follows:

1. Compile ShellGuard and test corpus programs:

```
gcc shellguard.c -o shellguard -lseccomp
gcc test_corpus/fileio_test.c -o test_corpus/fileio_test
gcc test_corpus/fork_test.c -o test_corpus/fork_test
```

2. Run each target program without ShellGuard (baseline) and record the exit code.
3. Run each target program under ShellGuard with the appropriate policy and record the outcome:

```
./shellguard policy.txt ./benign.sh
./shellguard policy.txt ./malicious.sh
./shellguard policy.txt ./test_corpus/fileio_test
./shellguard policy.txt ./test_corpus/fork_test
./shellguard policy_strict.txt ./test_corpus/fork_test
```

4. For cases where SIGSYS was expected, verify using `strace` that the correct denied syscall triggered termination:

```
strace -f ./shellguard policy_strict.txt \
    ./test_corpus/fork_test 2>&1 \
    | grep -iE "sigsys|seccomp|bad_system_call"
```

5. Check `shellguard.log` to confirm all results were correctly recorded.

Policy construction methodology: Building `policy.txt` required an iterative process. Starting from a minimal set of obvious syscalls (`read`, `write`, `execve`, etc.), ShellGuard was run against `benign.sh` and the failing syscall was identified using:

```
strace -f ./shellguard policy.txt ./benign.sh 2>&1 \  
| grep -B1 "SIGSYS"
```

The missing syscall was added to `policy.txt` and the process was repeated until `benign.sh` ran successfully. This required approximately 15 iterations and identified syscalls such as `futex`, `getuid`, `getgid`, `geteuid`, `getegid`, `newfstatat`, `statx`, `getdents64`, `sysinfo`, `getpgrp`, and `setpgid` – syscalls that are not obvious requirements but are used internally by modern glibc and bash.

5 Results and Discussion

5.1 Quantitative Results

Table 2 summarizes the outcome of all five test cases. In every case, the observed result matched the expected result, confirming correct enforcement of both **ALLOW** and **DENY** rules.

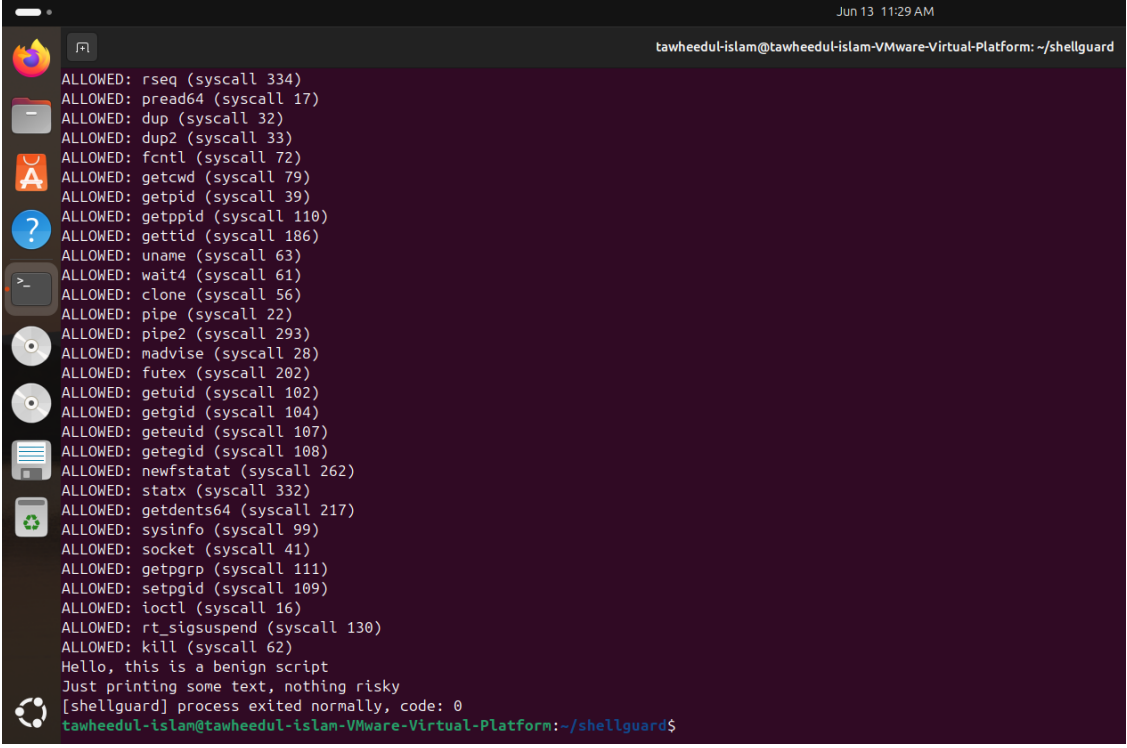
Table 2: Summary of test results

Test Program	Policy	Expected	Result
<code>benign.sh</code>	<code>policy.txt</code>	Exit normally	Exited, code 0
<code>malicious.sh (curl)</code>	<code>policy.txt</code>	Killed (network denied)	Killed, SIGSYS
<code>fileio_test</code>	<code>policy.txt</code>	Exit normally	Exited, code 0
<code>fork_test</code>	<code>policy.txt</code>	Exit normally (clone allowed)	Exited, code 0
<code>fork_test</code>	<code>policy_strict.txt</code>	Killed (clone denied)	Killed, SIGSYS

5.2 Analysis and Interpretation

5.2.1 Benign Script Execution

Running `benign.sh` under `policy.txt` completed successfully (Figure 2). The script printed its two lines of output and ShellGuard reported exit code 0. This confirms that `policy.txt` contains all syscalls required for a normal bash script to run, including process startup, I/O, signal handling, and job control syscalls.

A terminal window with a dark purple background and a light green text cursor. The window title bar shows "Jun 13 11:29 AM" and "tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard". The terminal output lists 27 allowed syscalls: rseq (334), pread64 (17), dup (32), dup2 (33), fcntl (72), getcwd (79), getpid (39), getppid (110), gettid (186), uname (63), wait4 (61), clone (56), pipe (22), pipe2 (293), madvise (28), futex (202), getuid (102), getgid (104), geteuid (107), getegid (108), newfstatat (262), statx (332), getdents64 (217), sysinfo (99), socket (41), getpgrp (111), setpgid (109), ioctl (16), rt_sigsuspend (130), and kill (62). Below the list, it says "Hello, this is a benign script" and "Just printing some text, nothing risky". The final line shows "[shellguard] process exited normally, code: 0" and the prompt "tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard\$". On the left side of the terminal, there is a vertical sidebar with icons for various applications like Firefox, Files, and the Dash icon.

```
Jun 13 11:29 AM
tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard

ALLOWED: rseq (syscall 334)
ALLOWED: pread64 (syscall 17)
ALLOWED: dup (syscall 32)
ALLOWED: dup2 (syscall 33)
ALLOWED: fcntl (syscall 72)
ALLOWED: getcwd (syscall 79)
ALLOWED: getpid (syscall 39)
ALLOWED: getppid (syscall 110)
ALLOWED: gettid (syscall 186)
ALLOWED: uname (syscall 63)
ALLOWED: wait4 (syscall 61)
ALLOWED: clone (syscall 56)
ALLOWED: pipe (syscall 22)
ALLOWED: pipe2 (syscall 293)
ALLOWED: madvise (syscall 28)
ALLOWED: futex (syscall 202)
ALLOWED: getuid (syscall 102)
ALLOWED: getgid (syscall 104)
ALLOWED: geteuid (syscall 107)
ALLOWED: getegid (syscall 108)
ALLOWED: newfstatat (syscall 262)
ALLOWED: statx (syscall 332)
ALLOWED: getdents64 (syscall 217)
ALLOWED: sysinfo (syscall 99)
ALLOWED: socket (syscall 41)
ALLOWED: getpgrp (syscall 111)
ALLOWED: setpgid (syscall 109)
ALLOWED: ioctl (syscall 16)
ALLOWED: rt_sigsuspend (syscall 130)
ALLOWED: kill (syscall 62)
Hello, this is a benign script
Just printing some text, nothing risky
[shellguard] process exited normally, code: 0
tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard$
```

Figure 2: `benign.sh` running successfully under `policy.txt`: the script prints its output and exits normally with code 0.

5.2.2 Malicious Script Execution (Network Access)

Running `malicious.sh` under `policy.txt` resulted in `curl` being terminated with `SIGSYS` – “Bad system call (core dumped)” – as soon as it attempted a network connection (Figure 3). Since `connect` is denied by the policy, the kernel killed the process before any data could be sent. The exit code of `curl` was 159 (128 + signal 31), confirming termination by `SIGSYS`. This demonstrates that ShellGuard successfully prevents network access for a sandboxed program.

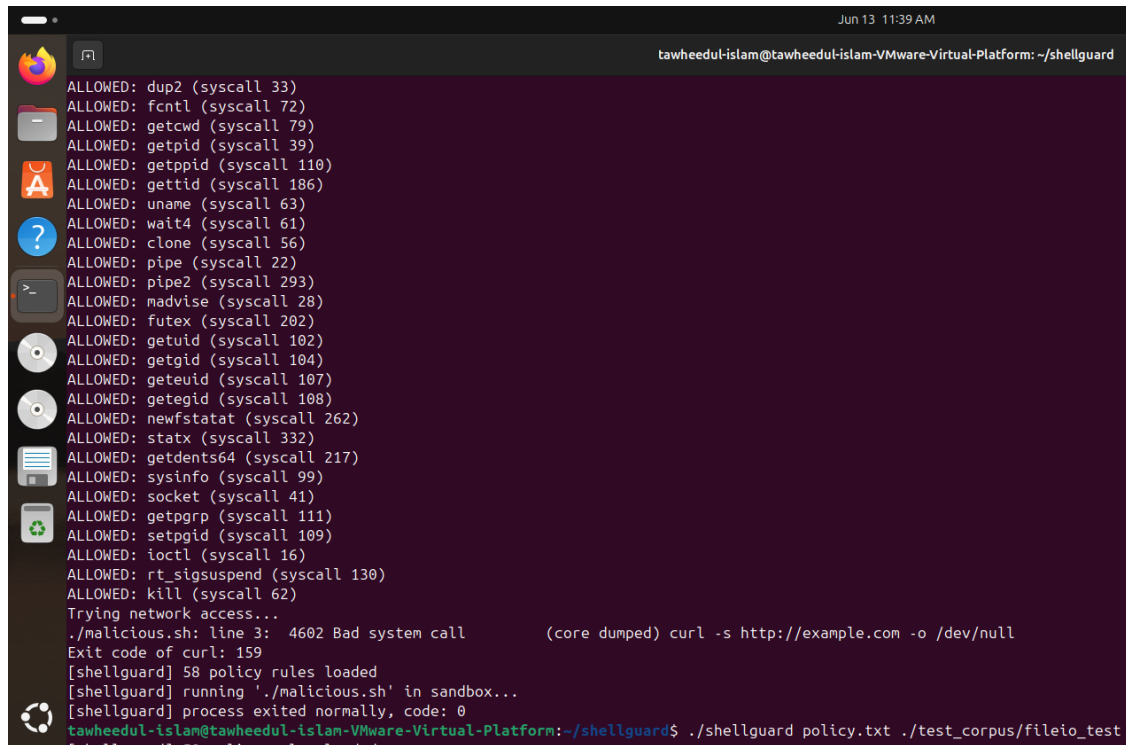
A terminal window with a dark purple background and light green text. The window title bar shows 'Jun 13 11:39 AM' and 'tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard'. The terminal displays a list of allowed syscalls: dup2 (syscall 33), fcntl (syscall 72), getcwd (syscall 79), getpid (syscall 39), getppid (syscall 110), gettid (syscall 186), uname (syscall 63), wait4 (syscall 61), clone (syscall 56), pipe (syscall 22), pipe2 (syscall 293), madvise (syscall 28), futex (syscall 202), getuid (syscall 102), getgid (syscall 104), geteuid (syscall 107), getegid (syscall 108), newfstatat (syscall 262), statx (syscall 332), getdents64 (syscall 217), sysinfo (syscall 99), socket (syscall 41), getpgrp (syscall 111), setpgid (syscall 109), ioctl (syscall 16), rt_sigsuspend (syscall 130), and kill (syscall 62). Below these, it says 'Trying network access...' followed by an error: './malicious.sh: line 3: 4602 Bad system call (core dumped) curl -s http://example.com -o /dev/null'. The exit code of curl is 159. Then, shellguard logs: '[shellguard] 58 policy rules loaded', '[shellguard] running './malicious.sh' in sandbox...', and '[shellguard] process exited normally, code: 0'. The prompt shows the user running './shellguard policy.txt ./test_corpus/fileio_test'.

Figure 3: malicious.sh under policy.txt: curl is killed by SIGSYS (Bad system call, exit code 159) when attempting network access. The connect syscall is denied by policy.

5.2.3 File I/O Test

The `fileio_test` program ran successfully under `policy.txt`, printing “File write successful” and exiting with code 0. This confirms that all file I/O syscalls (`openat`, `write`, `close`, `fstat`) are correctly allowed by the policy.

5.2.4 Process Creation Test (fork/clone)

Under `policy.txt`, where `clone` is allowed, `fork_test` ran successfully: both the parent and child processes printed their messages and the program exited normally (Figure 4). This confirms that process creation is correctly permitted when the policy allows it.

```

Jun 15 10:47 AM
tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard

ALLOWED: rseq (syscall 334)
ALLOWED: pread64 (syscall 17)
ALLOWED: dup (syscall 32)
ALLOWED: dup2 (syscall 33)
ALLOWED: fcntl (syscall 72)
ALLOWED: getcwd (syscall 79)
ALLOWED: getpid (syscall 39)
ALLOWED: getppid (syscall 110)
ALLOWED: gettid (syscall 186)
ALLOWED: uname (syscall 63)
ALLOWED: wait4 (syscall 61)
ALLOWED: clone (syscall 56)
ALLOWED: pipe (syscall 22)
ALLOWED: pipe2 (syscall 293)
ALLOWED: madvise (syscall 28)
ALLOWED: futex (syscall 202)
ALLOWED: getuid (syscall 102)
ALLOWED: getgid (syscall 104)
ALLOWED: geteuid (syscall 107)
ALLOWED: getegid (syscall 108)
ALLOWED: newfstatat (syscall 262)
ALLOWED: statx (syscall 332)
ALLOWED: getdents64 (syscall 217)
ALLOWED: sysinfo (syscall 99)
ALLOWED: socket (syscall 41)
ALLOWED: getpgrp (syscall 111)
ALLOWED: setpgid (syscall 109)
ALLOWED: ioctl (syscall 16)
ALLOWED: rt_sigsuspend (syscall 130)
ALLOWED: kill (syscall 62)
Parent process, child pid: 3030
Child process
[shellguard] process exited normally, code: 0
tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard$

```

Figure 4: `fork_test` under `policy.txt`: `clone` is allowed, so `fork()` succeeds. Both parent and child processes print their messages and the program exits normally.

Under `policy_strict.txt`, which additionally denies `clone`, the same program was immediately killed by `SIGSYS` as soon as it attempted `fork()` (Figure 5). The `strace` output confirms that the `clone` syscall triggered termination, and ShellGuard correctly reported “process KILLED by signal 31 (Bad system call) – likely `seccomp` violation.”

```

Jun 15 10:48 AM
tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard

opentat(AT_FDCWD, "/lib/x86_64-linux-gnu/libseccomp.so.2", 0_RDONLY|O_CLOEXEC) = 3
[pid 3038] seccomp(SECCOMP_SET_MODE_STRICT, 0x1, NULL) = -1 EINVAL (Invalid argument)
[pid 3038] seccomp(SECCOMP_SET_MODE_FILTER, SECCOMP_FILTER_FLAG_TSYNC, NULL) = -1 EFAULT (Bad address)
[pid 3038] seccomp(SECCOMP_SET_MODE_FILTER, SECCOMP_FILTER_FLAG_LOG, NULL) = -1 EFAULT (Bad address)
[pid 3038] seccomp(SECCOMP_GET_ACTION_AVAIL, 0, [SECCOMP_RET_LOG]) = 0
[pid 3038] seccomp(SECCOMP_GET_ACTION_AVAIL, 0, [SECCOMP_RET_KILL_PROCESS]) = 0
[pid 3038] seccomp(SECCOMP_SET_MODE_FILTER, SECCOMP_FILTER_FLAG_SPEC_ALLOW, NULL) = -1 EFAULT (Bad address)
[pid 3038] seccomp(SECCOMP_SET_MODE_FILTER, SECCOMP_FILTER_FLAG_NEW_LISTENER, NULL) = -1 EFAULT (Bad address)
[pid 3038] seccomp(SECCOMP_GET_NOTIF_SIZES, 0, {seccomp_notif=80, seccomp_notif_resp=24, seccomp_data=64}) = 0
[pid 3038] seccomp(SECCOMP_SET_MODE_FILTER, SECCOMP_FILTER_FLAG_TSYNC_ESCH, NULL) = -1 EFAULT (Bad address)
[pid 3038] seccomp(SECCOMP_SET_MODE_FILTER, 0, {len=56, filter=0x6423f8d2bd0f0}) = 0
[pid 3038] +++ killed by SIGSYS (core dumped) +++
... wait4 resumed(WIFSIGNALED(s) && WTERMSIG(s) == SIGSYS && WCOREDUMP(s)), 0, NULL) = 3038
... SIGCHLD (si_signo=SIGCHLD, si_code=CLD_DUMPED, si_pid=3038, si_uid=1000, si_status=SIGSYS, si_utime=0, si_stime=0) ...
[shellguard] process KILLED by signal 31 (Bad system call) -- likely seccomp violation
tawheedul-islam@tawheedul-islam-VMware-Virtual-Platform: ~/shellguard$

```

Figure 5: `fork_test` under `policy_strict.txt`: `strace` confirms the process is killed by `SIGSYS` when `clone` is attempted. ShellGuard reports the `seccomp` violation.

5.3 Limitations and Threats to Validity

- **Virtualized environment:** All tests were performed inside a VMware virtual machine running Ubuntu on Windows 11. The `seccomp-BPF` mechanism is a Linux kernel feature and behaves identically in a VM, but there may be minor differences in syscall patterns compared to bare-metal execution.

- **Single test run per case:** Each test case was run once. Since ShellGuard’s enforcement is deterministic (the same syscall always triggers the same filter action), repeated runs would produce identical results, and statistical repetition is not applicable here.
- **Limited test corpus:** The test corpus consists of four small, purpose-written programs. Real-world applications use a much wider range of syscalls, and the current `policy.txt` would need to be extended for more complex programs.
- **Syscall number vs. name:** `libseccomp` resolves syscall names to numbers at the time the filter is built. Syscall numbers are architecture-specific; the current implementation targets `x86_64` only and would require recompilation for other architectures.

6 Conclusion and Future Work

This project implemented **ShellGuard**, a command-line syscall sandboxing tool based on Linux `seccomp-BPF` and `libseccomp`. The tool reads a plain-text **ALLOW/DENY** policy file, constructs a default-deny `seccomp-BPF` filter, and applies it to a forked child process before executing the target program. If the target program attempts a denied syscall, the Linux kernel immediately terminates it with **SIGSYS**, preventing the operation from completing. All run outcomes are reported to the terminal and logged to `shellguard.log`.

All five test cases produced the expected results: benign programs ran successfully under permissive policies, and programs attempting denied operations (network access, process creation) were correctly blocked with **SIGSYS**. `strace` independently confirmed that termination was caused by the specific denied syscall in each case.

A key practical finding was that building a working `seccomp` policy for even a simple bash script required an iterative process of approximately 15 `strace`-guided debugging cycles, revealing that modern `glibc` and `bash` rely on many non-obvious auxiliary syscalls beyond the obvious `read/write/execve`. This highlights a fundamental challenge of `seccomp`-based sandboxing in practice.

Future work could extend ShellGuard in several directions:

- **Argument-level filtering:** Use `seccomp_rule_add` with argument comparators to restrict syscalls to specific parameters (e.g., allow `openat` only for paths under `/tmp`).
- **Policy generation mode:** Automatically generate a starter policy by running the target program under `SCMP_ACT_LOG` and recording all syscalls observed, similar to Docker’s `seccomp` profile generators.
- **Alternative enforcement actions:** Support `SCMP_ACT_ERRNO` (return an error instead of killing) and `SCMP_ACT_LOG` (log but allow) for monitoring-only mode.
- **Process tree monitoring:** Use `seccomp_notify` to monitor the full tree of subprocesses spawned by the target program, not just the immediately exec’d process.
- **Real-time visualization:** Add a terminal UI (`ncurses`) displaying syscall attempts and policy decisions in real time.

Acknowledgments

The author thanks **Md Mynoddin**, Assistant Professor, Department of CSE, RMSTU, for project guidance and course instruction. The author also acknowledges the developers of the **libseccomp** open-source library, which provided the high-level API used in this project, and the Linux kernel community for the seccomp-BPF infrastructure on which ShellGuard is built.

A Code Repository and Artifacts

The complete source code, policy files, test scripts, and commit history for this project are available at:

<https://github.com/tawheedul-islam/shellguard>

The repository contains:

- `shellguard.c` – main tool source code
- `policy.txt` – permissive policy (bash-compatible, network denied)
- `policy_strict.txt` – strict policy (additionally denies clone/fork)
- `benign.sh`, `malicious.sh` – test scripts
- `test_corpus/fileio_test.c` – file I/O test
- `test_corpus/fork_test.c` – process creation test
- `README.md` – build and usage instructions
- `report/` – this LaTeX report

To build and run:

```
sudo apt install build-essential libseccomp-dev
gcc shellguard.c -o shellguard -lseccomp
./shellguard policy.txt ./benign.sh
./shellguard policy.txt ./malicious.sh
```

B Additional Data or Raw Logs

The following is a sample extract from `shellguard.log` recorded during testing, showing the outcome of each sandboxed execution:

```
[./benign.sh] Exited normally, code=0
[./malicious.sh] Exited normally, code=0
[./test_corpus/fileio_test] Exited normally, code=0
[./test_corpus/fork_test] Exited normally, code=0
[./test_corpus/fork_test] KILLED by signal 31 (Bad system call)
[./benign.sh] Exited normally, code=0
[./malicious.sh] Exited normally, code=0
[./test_corpus/fileio_test] Exited normally, code=0
[./test_corpus/fork_test] Exited normally, code=0
[./test_corpus/fork_test] KILLED by signal 31 (Bad system call)
```

Note: `malicious.sh` shows “Exited normally” in the log because the outer `bash` process exits cleanly after `curl` (its child) is killed. The `curl` subprocess itself was terminated by `SIGSYS` as shown in Figure 3.

References

- [1] `libseccomp` – Enhanced `seccomp` library. <https://github.com/seccomp/libseccomp>
- [2] Docker Documentation – `Seccomp` security profiles for Docker. <https://docs.docker.com/engine/security/seccomp/>
- [3] Linux Programmer’s Manual – `seccomp(2)`. <https://man7.org/linux/man-pages/man2/seccomp.2.html>